

▼ 1. Instalacion de librerias y dependencias

```
# Instalar mongo

!pip install pymongo
!pip install kagglehub pandas pymongo
from pymongo import MongoClient
from pymongo.server_api import ServerApi
from random import randint, choice, uniform
from datetime import datetime
from datetime import datetime, timedelta
import kagglehub
import os
import pandas as pd
import math
import pprint
import time

Requirement already satisfied: pymongo in /usr/local/lib/python3.12/dist-packages (4.17.0)
Requirement already satisfied: dnspython<3.0.0,>=2.6.1 in /usr/local/lib/python3.12/dist-packages (from pymongo) (2.8.0)
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (1.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: pymongo in /usr/local/lib/python3.12/dist-packages (4.17.0)
Requirement already satisfied: kagglesdk<1.0,>=0.1.22 in /usr/local/lib/python3.12/dist-packages (from kagglehub) (0.1.23)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (26.2)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: dnspython<3.0.0,>=2.6.1 in /usr/local/lib/python3.12/dist-packages (from pymongo) (2.8.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.22->kagglehub) (5.29.6)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2026.5.20)
```

▼ 2. Carga de datos a la base de datos MongoDB

```
# Dataset de datos

path = kagglehub.dataset_download("olistbr/brazilian-e-commerce")
print("Path to dataset files:", path)

print(os.listdir(path))
products_df = pd.read_csv(os.path.join(path, "olist_products_dataset.csv"))
items_df = pd.read_csv(os.path.join(path, "olist_order_items_dataset.csv"))
reviews_df = pd.read_csv(os.path.join(path, "olist_order_reviews_dataset.csv"))
orders_df = pd.read_csv(os.path.join(path, "olist_orders_dataset.csv"))
sellers_df = pd.read_csv(os.path.join(path, "olist_sellers_dataset.csv"))
customers_df = pd.read_csv(os.path.join(path, "olist_customers_dataset.csv"))
category_translation_df = pd.read_csv(os.path.join(path, "product_category_name_translation.csv"))

print("products:", products_df.shape)
print("items:", items_df.shape)
print("reviews:", reviews_df.shape)
print("orders:", orders_df.shape)
print("sellers:", sellers_df.shape)
print("customers:", customers_df.shape)
print("categories:", category_translation_df.shape)

Using Colab cache for faster access to the 'brazilian-e-commerce' dataset.
Path to dataset files: /kaggle/input/brazilian-e-commerce
['olist_customers_dataset.csv', 'olist_sellers_dataset.csv', 'olist_order_reviews_dataset.csv', 'olist_order_items_dataset.csv', 'olist_products_dataset.csv', 'olist_geolocation_dataset.csv', 'product_category_name_translation.csv', 'olist_orders_dataset.csv', 'olist_order_payments_dataset.csv']
products: (32951, 9)
items: (12059, 7)
reviews: (99224, 7)
orders: (99441, 8)
sellers: (3895, 4)
customers: (99441, 5)
categories: (71, 2)
```

```
# Traducir informacion del portuges a ingles

products_df = products_df.merge(
    category_translation_df,
    on="product_category_name",
    how="left"
)

products_df["category"] = products_df["product_category_name_english"].fillna(
    products_df["product_category_name"]
)

products_df[["product_id", "product_category_name", "category"]].head()

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'product_category_name_english'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
      3810         ):
      3811             raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
      3813         except TypeError:
      3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'product_category_name_english'
```

Next steps: [Explain error](#)

```
# Métricas del dataset product_catalog

# Métricas de ventas por producto
sales_metrics = items_df.groupby("product_id").agg(
    total_units_sold=("order_item_id", "count"),
    avg_price=("price", "mean"),
    min_price=("price", "min"),
    max_price=("price", "max"),
    total_revenue=("price", "sum"),
    seller_id=("seller_id", "first")
).reset_index()

# Relacionar reviews con productos usando order_id
reviews_items = reviews_df.merge(
    items_df[["order_id", "product_id"]],
    on="order_id",
    how="left"
)

review_metrics = reviews_items.groupby("product_id").agg(
    average_rating=("review_score", "mean"),
    reviews_count=("review_id", "count")
).reset_index()

# Unir métricas al catálogo
catalog_df = products_df.merge(
    sales_metrics,
    on="product_id",
    how="left"
).merge(
    review_metrics,
    on="product_id",
    how="left"
)

catalog_df.head()
```

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm	product_category_name_english_x	category	product_category_name_english_y	total_units_sold	avg_price	min_price	max_price	total_revenue	seller_id	average
0	1e9e8ef04dbcf4541ed26657ea517e5	perfumaria	40.0	287.0	1.0	225.0	16.0	10.0	14.0	perfumery	perfumery	perfumery	1	10.91	10.91	10.91	10.91	5670f4db5b62c43d542e1b2d56b0c77c	
1	3aa071139cb16b67ca9e5dea841aaa2f	artes	44.0	276.0	1.0	1000.0	30.0	18.0	20.0	art	art	art	1	248.00	248.00	248.00	248.00	b561927807645834b59ef0d16ba55a24	
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	1.0	154.0	18.0	9.0	15.0	sports_leisure	sports_leisure	sports_leisure	1	79.80	79.80	79.80	79.80	7b07b3c7487f0ea825f6d775abd655b	
3	cef67bce19066a932b7673e239eb23d	bebés	27.0	261.0	1.0	371.0	26.0	4.0	26.0	baby	baby	baby	1	112.30	112.30	112.30	112.30	c510bc1718f0f2961eaa42a23330681a	
4	9dc1a7de274448489c219cf195d0b71	utilidades_domesticas	37.0	402.0	4.0	625.0	20.0	17.0	13.0	housewares	housewares	housewares	1	37.90	37.90	37.90	37.90	0be8ff43f22e456b4e0371b2245e4d01	

Next steps: [Generate code with catalog_df](#) [New interactive sheet](#)

```
# Ajustar a nuestro modelos de base de datos para seller region

catalog_df = catalog_df.merge(
    sellers_df[["seller_id", "seller_state", "seller_city"]],
    on="seller_id",
    how="left"
)

catalog_df["seller_region"] = catalog_df["seller_state"].fillna("UNKNOWN")
catalog_df["seller_city"] = catalog_df["seller_city"].fillna("UNKNOWN")

catalog_df.head()
```

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm	product_category_name_english_x	...	avg_price	min_price	max_price	total_revenue	seller_id	average_rating	reviews_count	seller_state	seller_city	seller_re
0	1e9e8ef04dbcf4541ed26657ea517e5	perfumaria	40.0	287.0	1.0	225.0	16.0	10.0	14.0	perfumery	...	10.91	10.91	10.91	10.91	5670f4db5b62c43d542e1b2d56b0c77c	5.0	1.0	SP	sao paulo	
1	3aa071139cb16b67ca9e5dea841aaa2f	artes	44.0	276.0	1.0	1000.0	30.0	18.0	20.0	art	...	248.00	248.00	248.00	248.00	b561927807645834b59ef0d16ba55a24	5.0	1.0	SP	sao paulo	
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	1.0	154.0	18.0	9.0	15.0	sports_leisure	...	79.80	79.80	79.80	79.80	7b07b3c7487f0ea825f6d775abd655b	5.0	1.0	SP	sao paulo	
3	cef67bce19066a932b7673e239eb23d	bebés	27.0	261.0	1.0	371.0	26.0	4.0	26.0	baby	...	112.30	112.30	112.30	112.30	c510bc1718f0f2961eaa42a23330681a	1.0	1.0	SP	indaiatuba	
4	9dc1a7de274448489c219cf195d0b71	utilidades_domesticas	37.0	402.0	4.0	625.0	20.0	17.0	13.0	housewares	...	37.90	37.90	37.90	37.90	0be8ff43f22e456b4e0371b2245e4d01	5.0	1.0	SP	sao paulo	

5 rows x 23 columns

```
# Configurar acceso

uri = "mongodb+srv://ecommyf_user:Y6xLoVUpkST8EPz8@ecommyf.a9hp3x7.mongodb.net/?appName=Ecommyf"
client = MongoClient(uri, server_api=ServerApi('1'))

# Verificar conexión a mongo
try:
    client.admin.command('ping')
    print("Estas conectado a mongo ecommyf !!!")
except Exception as e:
    print(e)

Estas conectado a mongo ecommyf !!!
```

```
# Trasformación de los datos para MongoDB product_catalog

def safe_value(value, default=None):
    if pd.isna(value):
        return default
    if isinstance(value, float) and math.isnan(value):
        return default
    return value

def build_product_document(row):
    product_id = row["product_id"]
```



```
category = safe_value(row.get("category"), "unknown")

price = safe_value(row.get("avg_price"), 0)
total_units_sold = safe_value(row.get("total_units_sold"), 0)
average_rating = safe_value(row.get("average_rating"), 0)
reviews_count = safe_value(row.get("reviews_count"), 0)
total_revenue = safe_value(row.get("total_revenue"), 0)

return {
    "id": str(product_id),
    "name": f"({category}) product",
    "description": f"Producto de la categoria {category} importado desde el dataset Olist Brazilian E-Commerce.",
    "category": str(category),
    "price": round(float(price), 2) if price else 0,
    "currency": "BRL",
    "seller_id": str(safe_value(row.get("seller_id"), "UNKNOWN")),
    "seller_region": str(safe_value(row.get("seller_region"), "UNKNOWN")),
    "seller_city": str(safe_value(row.get("seller_city"), "UNKNOWN")),
    "status": "ACTIVE" if total_units_sold and total_units_sold > 0 else "INACTIVE",
    "attributes": {
        "weight_g": safe_value(row.get("product_weight_g"), 0),
        "length_cm": safe_value(row.get("product_length_cm"), 0),
        "height_cm": safe_value(row.get("product_height_cm"), 0),
        "width_cm": safe_value(row.get("product_width_cm"), 0),
        "photos_qty": safe_value(row.get("product_photos_qty"), 0),
        "name_length": safe_value(row.get("product_name_lenght"), 0),
        "description_length": safe_value(row.get("product_description_lenght"), 0)
    },
    "tags": [
        str(category).lower(),
        "olist",
        "ecommiify",
        "catalog"
    ],
    "images": [],
    "ratings": {
        "average": round(float(average_rating), 2) if average_rating else 0,
        "count": int(reviews_count) if reviews_count else 0
    },
    "metrics": {
        "total_units_sold": int(total_units_sold) if total_units_sold else 0,
        "views_count": int(total_units_sold * 10) if total_units_sold else 0,
        "conversion_rate": 0.05,
        "total_revenue": round(float(total_revenue), 2) if total_revenue else 0
    },
    "recent_reviews": [],
    "source": {
        "dataset": "olistbr/brazilian-ecommerce",
        "original_product_category": safe_value(row.get("product_category_name"), "unknown")
    },
    "created_at": datetime.now(),
    "updated_at": datetime.now()
}
```

Insertar los registros product_catalog

```
db = client["ecommiify_db"]

product_catalog = db["product_catalog"]

product_catalog.delete_many({})

product_documents = []

for _, row in catalog_df.iterrows():
    product_documents.append(build_product_document(row))

product_catalog.insert_many(product_documents)

print("Total productos insertados en product_catalog:", product_catalog.count_documents({}))
```

Total productos insertados en product_catalog: 32951

Crear product_reviews con estructura Ecommify

```
reviews_enriched = reviews_df.merge(
    items_df[["order_id", "product_id"]],
    on="order_id",
    how="left"
)

reviews_enriched = reviews_enriched.dropna(subset=["product_id"])

# Eliminar duplicados por review_id + product_id
reviews_enriched = reviews_enriched.drop_duplicates(
    subset=["review_id", "product_id"]
)

print("Total reseñas después de limpiar duplicados:", len(reviews_enriched))
```

Total reseñas después de limpiar duplicados: 102172

Insertar los registros product_reviews

```
product_reviews = db["product_reviews"]

product_reviews.delete_many({})

review_documents = []

for _, row in reviews_enriched.iterrows():
    review_documents.append({
        "_id": f"({row['review_id']})_{row['product_id']}",
        "review_id": str(row["review_id"]),
        "product_id": str(row["product_id"]),
        "order_id": str(row["order_id"]),
        "rating": int(row["review_score"]),
        "comment": str(row["review_comment_message"]) if pd.notna(row["review_comment_message"]) else "",
        "title": str(row["review_comment_title"]) if pd.notna(row["review_comment_title"]) else "",
        "verified_purchase": True,
        "created_at": pd.to_datetime(row["review_creation_date"]).to_pydatetime() if pd.notna(row["review_creation_date"]) else datetime.now(),
        "answer_timestamp": pd.to_datetime(row["review_answer_timestamp"]).to_pydatetime() if pd.notna(row["review_answer_timestamp"]) else None,
        "source": {
            "dataset": "olistbr/brazilian-ecommerce"
        }
    })

batch_size = 5000

for i in range(0, len(review_documents), batch_size):
    product_reviews.insert_many(review_documents[i:i + batch_size])

print("Total reseñas insertadas en product_reviews:", product_reviews.count_documents({}))
```

Total reseñas insertadas en product_reviews: 102172

Crear sellers

```
sellers = db["sellers"]

sellers.delete_many({})

seller_documents = []

for _, row in sellers_df.iterrows():
    seller_documents.append({
        "_id": str(row["seller_id"]),
        "seller_id": str(row["seller_id"]),
        "city": str(row["seller_city"]),
        "region": str(row["seller_state"]),
        "country": "BR",
        "active": True,
        "source": {
            "dataset": "olistbr/brazilian-ecommerce"
        },
        "created_at": datetime.now(),
        "updated_at": datetime.now()
    })

sellers.insert_many(seller_documents)

print("Total sellers insertados:", sellers.count_documents({}))
```

Total sellers insertados: 3095

Crear user_behavior desde órdenes

```
order_items_customers = items_df.merge(
    orders_df[["order_id", "customer_id", "order_purchase_timestamp"]],
    on="order_id",
    how="left"
)

order_items_customers.head()
```

	order_id	order_item_id		product_id		seller_id	shipping_limit_date	price	freight_value		customer_id	order_purchase_timestamp	
0	00010242f68c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a8c2083c13a61	48436dade18ac8b2bce089ec2a041202		2017-09-19 09:45:35	58.90	13.29	3ce436f183ef8e07877b285a838db11a		2017-09-13 08:59:02		
1	00018f77f0320c57190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7dcd04e1b6c2cb14352b383efc2d36		2017-05-03 11:05:13	239.90	19.93	f6dd3ec061db4e3987629fe6b26e5cce		2017-04-26 10:53:06		
2	000229ec398224ef6ca0657da4fc703e	1	c777355d19b72b67abbeef9d44f0fd	5b51032eddd242ad084c38acab88f23d		2018-01-18 14:48:30	199.00	17.87	6489a6e5e4333f3693df5ad4372dab6d3		2018-01-14 14:33:31		
3	000242acbc0f0a6daae931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c204		2018-08-15 10:10:18	12.99	12.79	d4eb8395e8cd431ee92fce009800c5a06		2018-08-08 10:00:35		
4	00042b26df9d7ce09dfabb4e55b4fd9	1	ac8c362308f930de030a5865e4e10089	df560393f3a51e74553ab94004ba5c87		2017-02-13 13:57:51	199.90	18.14	58db0b62d70206bf40e62cd34e84d795		2017-02-04 13:57:51		

Insertar user_behavior desde órdenes

```
user_behavior = db["user_behavior"]

user_behavior.delete_many({})

behavior_documents = []

for _, row in order_items_customers.iterrows():
    customer_id = str(row["customer_id"])
    product_id = str(row["product_id"])
    timestamp = pd.to_datetime(row["order_purchase_timestamp"]).to_pydatetime() if pd.notna(row["order_purchase_timestamp"]) else datetime.now()

    behavior_documents.append({
        "userId": customer_id,
        "period": timestamp.strftime("%Y-%m"),
        "events": [
            {
                "type": "VIEW_PRODUCT",
                "productId": product_id,
                "timestamp": timestamp
            },
            {
                "type": "ADD_TO_CART",
                "productId": product_id,
                "timestamp": timestamp
            },
            {
                "type": "PURCHASE",
                "productId": product_id,
                "timestamp": timestamp
            }
        ],
        "source": {
            "dataset": "olistbr/brazilian-ecommerce",
            "order_id": str(row["order_id"])
        },
        "created_at": datetime.now()
    })

batch_size = 5000

for i in range(0, len(behavior_documents), batch_size):
    user_behavior.insert_many(behavior_documents[i:i + batch_size])

print("Total documentos insertados en user_behavior:", user_behavior.count_documents({}))
```

Total documentos insertados en user_behavior: 112650

3. Retroalimentación del estado actual de la base de datos antes de optimización


```
# Listar colecciones de la base de datos

# Antes de crear los índices de optimización, se realizó una revisión
# general del estado de la base de datos 'ecommify.db'. Esta revisión permitió
# identificar las colecciones existentes y validar que el dataset Olist fue
# transformado correctamente a la estructura documental definida para Ecommify.

print("Base de datos actual:", db.name)

collections = db.list_collection_names()

print("Colecciones existentes:")
for collection in collections:
    print("-", collection)
```

```
Base de datos actual: ecommify_db
Colecciones existentes:
- product_reviews
- product_catalog
- user_behavior
- sellers
```

```
# Conteo de documentos por colección

# El conteo de documentos permite validar la carga inicial de datos desde el
# dataset Olist hacia el modelo documental de Ecommify. La colección
# 'product_catalog' representa el catálogo extendido de productos,
# 'product_reviews' almacena las reseñas completas, 'sellers' contiene
# información de vendedores y 'user_behavior' simula eventos de comportamiento
# de usuario derivados de las órdenes.

collections_to_check = [
    "product_catalog",
    "product_reviews",
    "sellers",
    "user_behavior"
]

print("Conteo de documentos por colección:")

for collection_name in collections_to_check:
    if collection_name in db.list_collection_names():
        count = db[collection_name].count_documents({})
        print(collection_name, ":", count)
    else:
        print(collection_name, ": no existe")
```

```
Conteo de documentos por colección:
product_catalog : 32951
product_reviews : 102172
sellers : 3095
user_behavior : 112650
```

```
# Visualizar estructura de un producto

# La colección 'product_catalog' conserva la estructura definida para Ecommify.
# Cada producto contiene campos comunes como '_id', 'name', 'category', 'price',
# 'seller_id', 'seller_region', 'ratings' y 'metrics'. Además, mantiene el
# subdocumento flexible 'attributes', que permite almacenar características
# variables del producto, como peso, dimensiones, cantidad de fotos y longitud
# de descripción.

sample_product = product_catalog.find_one()

print("Documento de ejemplo en product_catalog:")
pprint.pprint(sample_product)
```

```
Documento de ejemplo en product_catalog:
{'_id': '1e9e8ef6d4dceff454ed266570a517a5',
 'attributes': {'description_length': 287.0,
               'height_cm': 10.0,
               'length_cm': 16.0,
               'name_length': 40.0,
               'photos_qty': 1.0,
               'weight_g': 225.0,
               'width_cm': 14.0},
 'category': 'perfumery',
 'created_at': datetime.datetime(2026, 6, 16, 1, 4, 6, 286000),
 'currency': 'BRL',
 'description': 'Producto de la categoría perfumery importado desde el dataset '
               'Olist Brazilian E-Commerce.',
 'images': [],
 'metrics': {'conversion_rate': 0.85,
            'total_revenue': 10.91,
            'total_units_sold': 1,
            'view_count': 10},
 'name': 'perfumery product',
 'price': 10.91,
 'ratings': {'average': 5.0, 'count': 1},
 'recent_reviews': [],
 'seller_city': 'sao paulo',
 'seller_id': '5670f4db5b62c43d542e1b2d56b0cf7c',
 'seller_region': 'sp',
 'source': {'dataset': 'olistbr/brazilian-ecommerce',
            'original_product_category': 'perfumaria'},
 'status': 'ACTIVE',
 'tags': ['perfumery', 'olist', 'ecommify', 'catalog'],
 'updated_at': datetime.datetime(2026, 6, 16, 1, 4, 6, 286000)}
```

```
# Visualizar estructura de una reseña

# La colección 'product_reviews' almacena las reseñas completas asociadas a
# los productos. La relación lógica con 'product_catalog' se realiza mediante
# el campo 'product_id', que corresponde al identificador del producto. Esta
# decisión sigue una estrategia de referencinng, evitando que las reseñas
# completas crezcan dentro del documento principal del producto.

sample_review = product_reviews.find_one()

print("Documento de ejemplo en product_reviews:")
pprint.pprint(sample_review)
```

```
Documento de ejemplo en product_reviews:
{'_id': '7bc2406c110b926393aa56f80a40eba40_fd25ab760bfbb13c198fa3b4f1a0cd3',
 'answer_timestamp': datetime.datetime(2018, 1, 18, 21, 46, 59),
 'comment': '',
 'created_at': datetime.datetime(2018, 1, 18, 0, 0),
 'order_id': '73fc7af87114b39712e6da79b0a377eb',
 'product_id': 'fd25a0760bfbb13c198fa3b4f1a0cd3',
 'rating': 4,
 'review_id': '7bc2406c110b926393aa56f80a40eba40',
 'source': {'dataset': 'olistbr/brazilian-ecommerce'},
 'title': '',
 'verified_purchase': True}
```

```
# Visualizar estructura de un seller

# La colección 'sellers' almacena la información básica de los vendedores,
# incluyendo ciudad, región y país. Esta colección permite enriquecer el
# catálogo con información geográfica del vendedor y apoyar análisis
# regionales sobre distribución de productos.

if "sellers" in db.list_collection_names():
    sample_seller = db["sellers"].find_one()

    print("Documento de ejemplo en sellers:")
    pprint.pprint(sample_seller)

Documento de ejemplo en sellers:
{'_id': '3442f8959a84dea7ee197c632cb2df15',
 'active': True,
 'city': 'campinas',
 'country': 'BR',
 'created_at': datetime.datetime(2026, 6, 16, 1, 17, 0, 610000),
 'region': 'sp',
 'seller_id': '3442f8959a84dea7ee197c632cb2df15',
 'source': {'dataset': 'olistbr/brazilian-ecommerce'},
 'updated_at': datetime.datetime(2026, 6, 16, 1, 17, 0, 610000)}
```

```
# Visualizar estructura de comportamiento de usuario

# La colección 'user_behavior' fue construida a partir de órdenes reales del
# dataset Olist. Debido a que el dataset no incluye navegación explícita, se
# simularon eventos como 'VIEW_PRODUCT', 'ADD_TO_CART' y 'PURCHASE' a partir de
# compras reales. Esta colección representa el módulo analítico de
# comportamiento de usuarios dentro de Ecommify.

if "user_behavior" in db.list_collection_names():
    sample_behavior = db["user_behavior"].find_one()

    print("Documento de ejemplo en user_behavior:")
    pprint.pprint(sample_behavior)

Documento de ejemplo en user_behavior:
{'_id': ObjectId('6a30a4a77087093ba934bd50'),
 'created_at': datetime.datetime(2026, 6, 16, 1, 18, 14, 690000),
 'events': [{'productId': '4244733e06e7ecb4970a6e2683c13e61',
             'timestamp': datetime.datetime(2017, 9, 13, 8, 59, 2),
             'type': 'VIEW_PRODUCT'},
            {'productId': '4244733e06e7ecb4970a6e2683c13e61',
             'timestamp': datetime.datetime(2017, 9, 13, 8, 59, 2),
             'type': 'ADD_TO_CART'},
            {'productId': '4244733e06e7ecb4970a6e2683c13e61',
             'timestamp': datetime.datetime(2017, 9, 13, 8, 59, 2),
             'type': 'PURCHASE'}],
 'period': '2017-09',
 'source': {'dataset': 'olistbr/brazilian-ecommerce',
            'order_id': '08018242feb8c5a6d1ba2dd792cb16214'},
 'userId': '3ce436f3183e6e078770285a838d011a'}
```

Relaciones lógicas entre colecciones

Aunque MongoDB no utiliza relaciones con llaves foráneas como una base de datos relacional, el modelo de Ecommify mantiene relaciones lógicas entre colecciones mediante identificadores.

Colección origen	Campo	Colección destino	Campo relacionado	Tipo de relación
product_reviews	product_id	product_catalog	_id	Un producto tiene muchas reseñas
product_catalog	seller_id	sellers	_id	Un vendedor tiene muchos productos
user_behavior	events.productId	product_catalog	_id	Un usuario genera eventos sobre productos
user_behavior	userId	Clientes del dataset Olist	customer_id	Un usuario tiene múltiples eventos
product_catalog	seller_region	Análisis regional	region	Agrupación geográfica

Estas relaciones son utilizadas para consultas analíticas y aggregation pipelines. En particular, la relación entre `product_catalog` y `product_reviews` permite aplicar `$lookup` para enriquecer productos con sus reseñas.

```
# Métricas Generales

# Se calcularon métricas generales del catálogo para conocer el estado inicial
# de la base de datos antes de aplicar optimizaciones. Estas métricas incluyen
# total de productos, precio promedio, precio mínimo, precio máximo,
# calificación promedio, unidades vendidas y revenue acumulado.

pipeline_catalog_metrics = [
    {
        "$group": {
            "_id": None,
            "total_products": {"$sum": 1},
            "avg_price": {"$avg": "$price"},
            "min_price": {"$min": "$price"},
            "max_price": {"$max": "$price"},
            "avg_rating": {"$avg": "$ratings.average"},
            "total_units_sold": {"$sum": "$metrics.total_units_sold"},
            "total_revenue": {"$sum": "$metrics.total_revenue"}
        },
    },
    {
        "$project": {
            "_id": 0,
            "total_products": 1,
            "avg_price": {"$round": [{"$avg_price", 2}]},
            "min_price": {"$round": [{"$min_price", 2}]},
            "max_price": {"$round": [{"$max_price", 2}]},
            "avg_rating": {"$round": [{"$avg_rating", 2}]},
            "total_units_sold": 1,
            "total_revenue": {"$round": [{"total_revenue", 2}]}
        }
    }
]

catalog_metrics = list(product_catalog.aggregate(pipeline_catalog_metrics))
```



```
print("Métricas generales del catálogo:")
for item in catalog_metrics:
    pprint.pprint(item)
```

```
Métricas generales del catálogo:
{'avg_price': 145.3,
 'avg_rating': 4.03,
 'max_price': 6735.0,
 'min_price': 0.45,
 'total_products': 32951,
 'total_revenue': 13591643.7,
 'total_units_sold': 112850}
```

```
pipeline_category_distribution = [
    {
        "$group": {
            "_id": "$category",
            "total_products": {"$sum": 1},
            "avg_price": {"$avg": "$price"},
            "avg_rating": {"$avg": "$ratings.average"},
            "total_units_sold": {"$sum": "$metrics.total_units_sold"},
            "total_revenue": {"$sum": "$metrics.total_revenue"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "category": "$_id",
            "total_products": 1,
            "avg_price": {"$round": [{"$avg_price", 2}]},
            "avg_rating": {"$round": [{"$avg_rating", 2}]},
            "total_units_sold": 1,
            "total_revenue": {"$round": [{"$total_revenue", 2}]}
        }
    },
    {
        "$sort": {
            "total_products": -1
        }
    },
    {
        "$limit": 15
    }
]

category_distribution = list(product_catalog.aggregate(pipeline_category_distribution))

print("Top 15 categorías por cantidad de productos:")
for item in category_distribution:
    pprint.pprint(item)
```

```
{'avg_price': 98.56,
 'avg_rating': 4.06,
 'category': 'housewares',
 'total_products': 2335,
 'total_revenue': 632248.66,
 'total_units_sold': 6964}
{'avg_price': 152.58,
 'avg_rating': 4.02,
 'category': 'auto',
 'total_products': 3908,
 'total_revenue': 592720.11,
 'total_units_sold': 4235}
{'avg_price': 156.03,
 'avg_rating': 3.93,
 'category': 'computers_accessories',
 'total_products': 1639,
 'total_revenue': 911954.32,
 'total_units_sold': 7827}
{'avg_price': 133.28,
 'avg_rating': 4.14,
 'category': 'toys',
 'total_products': 1411,
 'total_revenue': 483946.6,
 'total_units_sold': 4117}
{'avg_price': 335.91,
 'avg_rating': 4.14,
 'category': 'watches_gifts',
 'total_products': 1329,
 'total_revenue': 1205005.68,
 'total_units_sold': 5991}
{'avg_price': 83.83,
 'avg_rating': 3.92,
 'category': 'telephony',
 'total_products': 1134,
 'total_revenue': 323667.53,
 'total_units_sold': 4545}
{'avg_price': 166.97,
 'avg_rating': 3.95,
 'category': 'baby',
 'total_products': 919,
 'total_revenue': 411764.89,
 'total_units_sold': 3065}
{'avg_price': 122.46,
 'avg_rating': 4.11,
 'category': 'perfumery',
 'total_products': 868,
 'total_revenue': 399124.87,
 'total_units_sold': 3419}
{'avg_price': 103.57,
 'avg_rating': 4.2,
 'category': 'stationery',
 'total_products': 849,
 'total_revenue': 230943.23,
 'total_units_sold': 2517}
{'avg_price': 85.75,
 'avg_rating': 4.1,
 'category': 'fashion_bags_accessories',
 'total_products': 849,
```

```
pipeline_region_distribution = [
    {
        "$group": {
            "_id": "$seller_region",
            "total_products": {"$sum": 1},
            "avg_price": {"$avg": "$price"},
            "avg_rating": {"$avg": "$ratings.average"},
            "total_units_sold": {"$sum": "$metrics.total_units_sold"},
            "total_revenue": {"$sum": "$metrics.total_revenue"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "seller_region": "$_id",
            "total_products": 1,
            "avg_price": {"$round": [{"$avg_price", 2}]},
            "avg_rating": {"$round": [{"$avg_rating", 2}]},
            "total_units_sold": 1,
            "total_revenue": {"$round": [{"$total_revenue", 2}]}
        }
    },
    {
        "$sort": {
            "total_products": -1
        }
    }
]

region_distribution = list(product_catalog.aggregate(pipeline_region_distribution))

print("Distribución por región del vendedor:")
for item in region_distribution:
    pprint.pprint(item)
```

```
Distribución por región del vendedor:
{'avg_price': 133.29,
 'avg_rating': 4.01,
 'seller_region': 'SP',
 'total_products': 2265,
 'total_revenue': 8740778.77,
 'total_units_sold': 80142}
{'avg_price': 168.5,
 'avg_rating': 4.08,
 'seller_region': 'PR',
 'total_products': 2889,
 'total_revenue': 1272061.09,
 'total_units_sold': 8853}
{'avg_price': 129.2,
 'avg_rating': 4.05,
 'seller_region': 'MG',
 'total_products': 2682,
 'total_revenue': 1008367.31,
 'total_units_sold': 8792}
{'avg_price': 224.24,
 'avg_rating': 4.09,
 'seller_region': 'R1',
 'total_products': 1444,
 'total_revenue': 861136.24,
 'total_units_sold': 4929}
{'avg_price': 174.79,
 'avg_rating': 4.07,
 'seller_region': 'SC',
 'total_products': 1306,
 'total_revenue': 618330.55,
 'total_units_sold': 4015}
{'avg_price': 194.29,
 'avg_rating': 4.16,
 'seller_region': 'RS',
 'total_products': 755,
 'total_revenue': 379071.07,
 'total_units_sold': 2201}
{'avg_price': 130.19,
 'avg_rating': 4.05,
 'seller_region': 'DP',
 'total_products': 359,
 'total_revenue': 96602.68,
 'total_units_sold': 875}
{'avg_price': 150.87,
 'avg_rating': 4.18,
 'seller_region': 'GO',
 'total_products': 125,
 'total_revenue': 70122.42,
 'total_units_sold': 542}
{'avg_price': 391.87,
 'avg_rating': 4.04,
 'seller_region': 'BA',
 'total_products': 124,
 'total_revenue': 205308.98,
 'total_units_sold': 641}
{'avg_price': 134.45,
 'avg_rating': 4.09,
 'seller_region': 'ES',
```

```
pipeline_region_distribution = [
    {
        "$group": {
            "_id": "$seller_region",
            "total_products": {"$sum": 1},
            "avg_price": {"$avg": "$price"},
            "avg_rating": {"$avg": "$ratings.average"},
            "total_units_sold": {"$sum": "$metrics.total_units_sold"},
            "total_revenue": {"$sum": "$metrics.total_revenue"}
        }
    },
    {
        "$project": {
            "_id": 0,
            "seller_region": "$_id",
            "total_products": 1,
            "avg_price": {"$round": [{"$avg_price", 2}]},
            "avg_rating": {"$round": [{"$avg_rating", 2}]},
            "total_units_sold": 1,
            "total_revenue": {"$round": [{"$total_revenue", 2}]}
        }
    },
    {
        "$sort": {
            "total_products": -1
        }
    }
]

region_distribution = list(product_catalog.aggregate(pipeline_region_distribution))

print("Distribución por región del vendedor:")
for item in region_distribution:
    pprint.pprint(item)
```



```
{ 'avg_price': 129.87,
  'avg_rating': 4.18,
  'seller_region': 'BO',
  'total_products': 225,
  'total_revenue': 78122.42,
  'total_units_sold': 542}
{'avg_price': 391.87,
  'avg_rating': 4.04,
  'seller_region': 'BA',
  'total_products': 124,
  'total_revenue': 285368.98,
  'total_units_sold': 641}
{'avg_price': 134.45,
  'avg_rating': 4.09,
  'seller_region': 'ES',
  'total_products': 95,
  'total_revenue': 47194.71,
  'total_units_sold': 369}
{'avg_price': 452.32,
  'avg_rating': 3.96,
  'seller_region': 'PE',
  'total_products': 61,
  'total_revenue': 98166.33,
  'total_units_sold': 458}
{'avg_price': 104.28,
  'avg_rating': 4.18,
  'seller_region': 'MT',
  'total_products': 61,
  'total_revenue': 12870.72,
  'total_units_sold': 145}
{'avg_price': 88.98,
  'avg_rating': 4.0,
  'seller_region': 'MA',
  'total_products': 44,
  'total_revenue': 36488.95,
  'total_units_sold': 465}
{'avg_price': 564.9,
  'avg_rating': 3.6,
  'seller_region': 'PB',
  'total_products': 29,
  'total_revenue': 17794.0,
  'total_units_sold': 39}
{'avg_price': 178.86,
  'avg_rating': 3.69,
  'seller_region': 'CE',
  'total_products': 29,
  'total_revenue': 21838.39,
  'total_units_sold': 98}
{'avg_price': 174.38,
  'avg_rating': 4.19,
  'seller_region': 'MS',
  'total_products': 27,
  'total_revenue': 8859.49,
  'total_units_sold': 47}
{'avg_price': 129.87,
  'avg_rating': 4.18,
  'seller_region': 'BO',
  'total_products': 225,
  'total_revenue': 78122.42,
  'total_units_sold': 542}
```

```
pipeline_review_metrics = [
    {
        "$group": {
            "_id": None,
            "total_reviews": {"$sum": 1},
            "avg_rating": {"$avg": "$rating"},
            "min_rating": {"$min": "$rating"},
            "max_rating": {"$max": "$rating"},
            "verified_reviews": {
                "$sum": {
                    "$cond": [{"$eq": ["$verified_purchase", True]}, 1, 0]
                }
            }
        },
        {
            "$project": {
                "_id": 0,
                "total_reviews": 1,
                "avg_rating": {"$round": ["$avg_rating", 2]},
                "min_rating": 1,
                "max_rating": 1,
                "verified_reviews": 1
            }
        }
    ]

review_metrics = list(product_reviews.aggregate(pipeline_review_metrics))

print("Métricas generales de reseñas:")
for item in review_metrics:
    pprint.pprint(item)

Métricas generales de reseñas:
{'avg_rating': 4.08,
 'max_rating': 5,
 'min_rating': 1,
 'total_reviews': 182172,
 'verified_reviews': 182172}
```

```
pipeline_rating_distribution = [
    {
        "$group": {
            "_id": "$rating",
            "total_reviews": {"$sum": 1}
        },
        {
            "$project": {
                "_id": 0,
                "rating": "$_id",
                "total_reviews": 1
            }
        },
        {
            "$sort": {
                "rating": 1
            }
        }
    ]

rating_distribution = list(product_reviews.aggregate(pipeline_rating_distribution))

print("Distribución de reseñas por calificación:")
for item in rating_distribution:
    pprint.pprint(item)

Distribución de reseñas por calificación:
{'rating': 1, 'total_reviews': 11899}
{'rating': 2, 'total_reviews': 3376}
{'rating': 3, 'total_reviews': 8522}
{'rating': 4, 'total_reviews': 19588}
{'rating': 5, 'total_reviews': 58787}
```

```
pipeline_top_revenue = [
    {
        "$sort": {
            "metrics.total_revenue": -1
        },
        {
            "$project": {
                "_id": 1,
                "name": 1,
                "category": 1,
                "seller_region": 1,
                "price": 1,
                "ratings.average": 1,
                "metrics.total_units_sold": 1,
                "metrics.total_revenue": 1
            }
        },
        {
            "$limit": 10
        }
    ]

top_revenue_products = list(product_catalog.aggregate(pipeline_top_revenue))

print("Top 10 productos por revenue:")
for item in top_revenue_products:
    pprint.pprint(item)

{'_id': '54738b2485095c36a24e33960858',
 'metrics': {'total_revenue': 54738.2, 'total_units_sold': 156},
 'name': 'health_beauty product',
 'price': 358.83,
 'ratings': {'average': 4.32},
 'seller_region': 'PR',
 'category': 'computers',
 'metrics': {'total_revenue': 48899.34, 'total_units_sold': 35},
 'name': 'computers product',
 'price': 1397.12,
 'ratings': {'average': 4.57},
 'seller_region': 'BA',
 '_id': 'd1c427080a0f73f6b889a5c7c61f2ac4',
 'category': 'computers_accessories',
 'metrics': {'total_revenue': 47214.51, 'total_units_sold': 343},
 'name': 'computers_accessories product',
 'price': 137.65,
 'ratings': {'average': 4.19},
 'seller_region': 'MG',
 '_id': '99a4788cb2485095c36a24e33960858',
 'category': 'bed_bath_table',
 'metrics': {'total_revenue': 43025.56, 'total_units_sold': 488},
 'name': 'bed_bath_table product',
 'price': 88.17,
 'ratings': {'average': 3.9},
 'seller_region': 'SP',
 '_id': '3dd2a17168ec895c781a9191c1e95ad7',
 'category': 'computers_accessories',
 'metrics': {'total_revenue': 41882.6, 'total_units_sold': 274},
 'name': 'computers_accessories product',
 'price': 149.94,
 'ratings': {'average': 4.21},
 'seller_region': 'PE',
 '_id': '25c38557cf93876c5abdd5931f922db',
 'category': 'baby',
 'metrics': {'total_revenue': 38907.32, 'total_units_sold': 38},
 'name': 'baby product',
 'price': 1023.88,
 'ratings': {'average': 2.68},
 'seller_region': 'SP',
 '_id': '5f584031a1c75b73d6151be81eb05bdc9',
 'category': 'cool_stuff',
 'metrics': {'total_revenue': 37733.9, 'total_units_sold': 63},
 'name': 'cool_stuff product',
 'price': 598.95,
 'ratings': {'average': 4.56},
 'seller_region': 'SP',
 '_id': '53b36df67ebb7c41585e8d54d6772e08',
 'category': 'watches_gifts',
 'metrics': {'total_revenue': 37683.42, 'total_units_sold': 323},
 'name': 'watches_gifts product',
 'price': 116.67,
 'ratings': {'average': 4.19},
 'seller_region': 'SP',
 '_id': 'aca2eb700ba1a7b8ebd4e6814663af',
 'category': 'furniture_decor',
 'metrics': {'total_revenue': 37688.9, 'total_units_sold': 527},
 'name': 'furniture_decor product',
 'price': 71.7,
 'ratings': {'average': 4.19},
 'seller_region': 'SP'}
```

```
print("Índices actuales en product_catalog:")
for index in product_catalog.list_indexes():
    pprint.pprint(index)

print("\nÍndices actuales en product_reviews:")
for index in product_reviews.list_indexes():
    pprint.pprint(index)

if "user_behavior" in db.list_collection_names():
    print("\nÍndices actuales en user_behavior:")
    for index in db["user_behavior"].list_indexes():
        pprint.pprint(index)

Índices actuales en product_catalog:
SON([('v', 2), ('key', SON([('id', 1)]))], ('name', '_id_1'))
SON([('v', 2), ('key', SON([('category', 1)]))], ('name', 'category_1'))
SON([('v', 2), ('key', SON([('price', 1)]))], ('name', 'price_1'))
SON([('v', 2), ('key', SON([('tags', 1)]))], ('name', 'tags_1'))
SON([('v', 2), ('key', SON([('seller_id', 1)]))], ('name', 'seller_id_1'))
SON([('v', 2), ('key', SON([('seller_region', 1)]))], ('name', 'seller_region_1'))
SON([('v', 2), ('key', SON([('ratings.average', -1)]))], ('name', 'ratings.average-1'))
SON([('v', 2), ('key', SON([('metrics.total_units_sold', -1)]))], ('name', 'metrics.total_units_sold-1'))
SON([('v', 2), ('key', SON([('attributes.name', 1)]))], ('name', 'attributes.name_1'))
SON([('v', 2), ('key', SON([('category', 1), ('price', 1)]))], ('name', 'category_1_price_1'))
SON([('v', 2), ('key', SON([('category', 1), ('seller_region', 1), ('id', 'hashed')]))], ('name', 'category_1_seller_region_1_id_hashed'))
```



```
Indices actuales en product_reviews:
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
SON([{'v': 2}, {'key': SON([{'product_id': 1}])}], {'name': 'product_id'})
SON([{'v': 2}, {'key': SON([{'product_id': 'hashed'})}], {'name': 'product_id_hashed'})

Indices actuales en user_behavior:
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
SON([{'v': 2}, {'key': SON([{'userId': 'hashed'}, {'period': 1}])}], {'name': 'userId_hashed_period_1'})
```

Retroalimentación del estado inicial de la base de datos

Antes de implementar los índices de optimización, se realizó una revisión del estado de la base de datos `ecommi fy_db`. El dataset Olist Brazilian E-Commerce fue transformado al modelo documental de Ecommify, conservando las colecciones principales `product_catalog`, `product_reviews`, `sellers` y `user_behavior`.

La colección `product_catalog` representa el catálogo extendido de productos. Cada documento contiene campos comunes como identificador, categoría, precio, vendedor, región, calificaciones y métricas comerciales. También conserva el subdocumento `attributes`, que permite almacenar características variables del producto. La colección `product_reviews` almacena las reseñas completas y se relaciona lógicamente con `product_catalog` mediante el campo `product_id`.

La colección `sellers` almacena información geográfica de los vendedores y permite enriquecer el análisis regional del catálogo. La colección `user_behavior` fue construida a partir de órdenes reales del dataset y simula eventos de comportamiento como visualización de producto, adición al carrito y compra.

Como parte del diagnóstico, se calcularon métricas generales del catálogo, distribución de productos por categoría, distribución por región del vendedor, métricas de reseñas, distribución de calificaciones y productos con mayor revenue. Estas métricas permiten identificar patrones de consulta, posibles concentraciones de datos y campos candidatos para optimización.

Finalmente, se revisaron los índices existentes en las colecciones principales para establecer una línea base antes de crear índices compuestos, parciales y de texto. Esta retroalimentación permite justificar técnicamente las optimizaciones aplicadas en las siguientes secciones.

4. Crear índices de optimización sobre dataset real

En esta sección se implementan optimizaciones sobre la colección `product_catalog` y la colección `product_reviews` utilizando índices compuestos, índices parciales e índices de texto.

La estrategia principal para los índices compuestos se basa en la regla ESR:

- **E - Equality:** campos filtrados por igualdad.
- **S - Sort:** campos usados para ordenamiento.
- **R - Range:** campos usados con rangos.

La regla ESR permite ordenar los campos del índice de acuerdo con la forma en que se ejecutan las consultas. Primero se ubican los campos usados con igualdad, después los campos usados para ordenamiento y finalmente los campos usados para filtros de rango.

Además, se implementan índices parciales para subconjuntos relevantes de datos, como productos activos con reseñas suficientes y reseñas verificadas. Finalmente, se crea un índice de texto para permitir búsquedas full-text sobre nombre, descripción y etiquetas del producto.

```
# Funcion para evaluar la implementacion de los indices
def evaluate_query_productivity(label, cursor):
    explain_result = cursor.explain()
    stats = explain_result['executionStats']

    n_returned = stats.get("nReturned", 0)
    docs_examined = stats.get("totalDocsExamined", 0)
    keys_examined = stats.get("totalKeysExamined", 0)
    execution_time = stats.get("executionTimeMillis", 0)

    docs_per_returned = round(docs_examined / n_returned, 2) if n_returned > 0 else None
    keys_per_returned = round(keys_examined / n_returned, 2) if n_returned > 0 else None

    result = {
        "consulta": label,
        "nReturned": n_returned,
        "totalDocsExamined": docs_examined,
        "totalKeysExamined": keys_examined,
        "executionTimeMillis": execution_time,
        "docsPerReturned": docs_per_returned,
        "keysPerReturned": keys_per_returned
    }

    print("====", label, "====")
    pprint.pprint(result)

    print("\nPlan ganador:")
    pprint.pprint(explain_result["queryPlanner"]["winningPlan"])

    return result
```

```
index_evaluation_results = []

print("Base de datos:", db.name)

print("\nColecciones disponibles:")
for collection_name in db.list_collection_names():
    print("-", collection_name)

collections_to_check = [
    "product_catalog",
    "product_reviews",
    "user_behavior",
    "sellers"
]

for collection_name in collections_to_check:
    if collection_name in db.list_collection_names():
        print(collection_name, ":", db[collection_name].count_documents({}))
    else:
        print(collection_name, ": no existe")
```

```
Base de datos: ecommify_db

Colecciones disponibles:
- product_reviews
- product_catalog
- user_behavior
- sellers
product_catalog : 32951
product_reviews : 102272
user_behavior : 112650
sellers : 3095
```

```
for collection_name in collections_to_check:
    if collection_name in db.list_collection_names():
        print("\nIndices actuales en", collection_name)
        for index in db[collection_name].list_indexes():
            pprint.pprint(index)
```

```
Indices actuales en product_catalog
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
SON([{'v': 2}, {'key': SON([{'category': 1}])}], {'name': 'category_1'})
SON([{'v': 2}, {'key': SON([{'price': 1}])}], {'name': 'price_1'})
SON([{'v': 2}, {'key': SON([{'tags': 1}])}], {'name': 'tags_1'})
SON([{'v': 2}, {'key': SON([{'seller_id': 1}])}], {'name': 'seller_id_1'})
SON([{'v': 2}, {'key': SON([{'seller_region': 1}])}], {'name': 'seller_region_1'})
SON([{'v': 2}, {'key': SON([{'ratings_average': -1}])}], {'name': 'ratings_average_-1'})
SON([{'v': 2}, {'key': SON([{'metrics_total_units_sold': -1}])}], {'name': 'metrics_total_units_sold_-1'})
SON([{'v': 2}, {'key': SON([{'attributes_ram': 1}])}], {'name': 'attributes_ram_1'})
SON([{'v': 2}, {'key': SON([{'category': 1}, {'price': 1}])}], {'name': 'category_1_price_1'})
SON([{'v': 2}, {'key': SON([{'category': 1}, {'seller_region': 1}, {'_id': 'hashed'})}], {'name': 'category_1_seller_region_1_id_hashed'})
SON([{'v': 2}, {'key': SON([{'status': 1}, {'category': 1}, {'metrics_total_units_sold': -1}, {'price': 1}])}], {'name': 'idx_esr_status_category_units_price'})
SON([{'v': 2}, {'key': SON([{'category': 1}, {'ratings_average': -1}, {'price': 1}])}], {'name': 'idx_category_rating_price_esr'})
SON([{'v': 2}, {'key': SON([{'status': 1}])}], {'name': 'idx_partial_active_products'}, {'partialFilterExpression': SON([{'status': 'ACTIVE'})}])

Indices actuales en product_reviews
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
SON([{'v': 2}, {'key': SON([{'product_id': 1}])}], {'name': 'product_id_1'})
SON([{'v': 2}, {'key': SON([{'product_id': 'hashed'})}], {'name': 'product_id_hashed'})

Indices actuales en user_behavior
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
SON([{'v': 2}, {'key': SON([{'userId': 'hashed'}, {'period': 1}])}], {'name': 'userId_hashed_period_1'})
SON([{'v': 2}, {'key': SON([{'userId': 1}, {'events_timestamp': 1}, {'idx_user_timeline': 1}])}], {'name': 'idx_user_timeline'})
SON([{'v': 2}, {'key': SON([{'userId': 1}, {'period': 1}])}], {'name': 'idx_user_behavior_lookup'})

Indices actuales en sellers
SON([{'v': 2}, {'key': SON([{'_id': 1}])}], {'name': '_id'})
```

Implementacion de indices compuestos, parciales y texto

Coleccion product_catalog

```
# Optimización de product_catalog

query_catalog_esr = {
    "category": "bed_bath_table",
    "seller_region": "SP",
    "status": "ACTIVE",
    "price": {
        "$gte": 50,
        "$lte": 500
    }
}

cursor_catalog_esr_before = product_catalog.find(
    query_catalog_esr
).sort("ratings.average", -1).hint({"$natural": 1})

result = evaluate_query_productivity(
    "product_catalog - ESR antes del indice",
    cursor_catalog_esr_before
)

index_evaluation_results.append(result)

==== product_catalog - ESR antes del indice ====
{'consulta': 'product_catalog - ESR antes del indice',
 'docsPerReturned': 17.77,
 'executionTimeMillis': 45,
 'keysPerReturned': 0.0,
 'nReturned': 1854,
 'totalDocsExamined': 32951,
 'totalKeysExamined': 0}

Plan ganador:
{'inputStage': {'direction': 'forward',
 'filter': {'$and': [{'category': {'$eq': 'bed_bath_table'}},
 {'seller_region': {'$eq': 'SP'}},
 {'status': {'$eq': 'ACTIVE'}},
 {'price': {'$lte': 500}},
 {'price': {'$gte': 50}}]}},
 'stage': 'COLLSCAN'},
 'isCached': False,
 'memLimit': 33594432,
 'sortPattern': {'ratings.average': -1},
 'stage': 'SORT',
 'type': 'simple'}
```

```
product_catalog.drop_index("idx_pc_esr_category_region_status_rating_price")
```

```
product_catalog.create_index(
    [
        ("category", 1),
        ("seller_region", 1),
        ("status", 1),
        ("ratings.average", -1),
        ("price", 1)
    ],
    name="idx_pc_esr_category_region_status_rating_price"
)

print("Índice creado: idx_pc_esr_category_region_status_rating_price")

Índice creado: idx_pc_esr_category_region_status_rating_price
```

```
cursor_catalog_esr_after = product_catalog.find(
    query_catalog_esr
).sort("ratings.average", -1).hint("idx_pc_esr_category_region_status_rating_price")
```



```
result = evaluate_query_productivity(
    "product_catalog - ESR usando idx_pc_esr_category_region_status_rating_price",
    cursor_catalog_esr_after
)

index_evaluation_results.append(result)

==== product_catalog - ESR usando idx_pc_esr_category_region_status_rating_price ====
{'consulta': 'product_catalog - ESR usando '
            'idx_pc_esr_category_region_status_rating_price',
 'docsPerReturned': 1.0,
 'executionTimeMillis': 6,
 'keysPerReturned': 1.05,
 'mReturned': 1854,
 'totalDocsExamined': 1854,
 'totalKeysExamined': 1942}

Plan ganador:
{'inputStage': {'direction': 'forward',
                'indexBounds': {'category': [['bed_bath_table', '
                                'bed_bath_table']],
                                'price': [['$0, 500]]},
                                'ratings.average': [['MaxKey, MinKey']],
                                'seller_region': [['SP', 'SP']],
                                'status': [['ACTIVE', 'ACTIVE']]},
                'indexName': 'idx_pc_esr_category_region_status_rating_price',
                'indexVersion': 2,
                'isMultiKey': False,
                'isPartial': False,
                'isSparse': False,
                'isUnique': False,
                'keyPattern': {'category': 1,
                              'price': 1,
                              'ratings.average': -1,
                              'seller_region': 1,
                              'status': 1},
                'multikeyPaths': {'category': [],
                                  'price': [],
                                  'ratings.average': [],
                                  'seller_region': [],
                                  'status': []},
                'stage': 'IXSCAN'},
 'isCached': False,
 'stage': 'FETCH'}
```

```
# Índice ESR para productos más vendidos

query_catalog_top_sold = {
    "status": "ACTIVE",
    "category": "bed_bath_table",
    "price": {
        "$gte": 50,
        "$lte": 1000
    }
}

cursor_top_sold_before = product_catalog.find(
    query_catalog_top_sold
).sort("metrics.total_units_sold", -1).hint({"$natural": 1})

result = evaluate_query_productivity(
    "product_catalog - top vendidos antes del índice",
    cursor_top_sold_before
)

index_evaluation_results.append(result)
```

```
==== product_catalog - top vendidos antes del índice ====
{'consulta': 'product_catalog - top vendidos antes del índice',
 'docsPerReturned': 14.85,
 'executionTimeMillis': 29,
 'keysPerReturned': 0.0,
 'mReturned': 2219,
 'totalDocsExamined': 32951,
 'totalKeysExamined': 0}

Plan ganador:
{'inputStage': {'direction': 'forward',
                'filter': {'$and': [{'category': {'$eq': 'bed_bath_table'}},
                                    {'status': {'$eq': 'ACTIVE'}},
                                    {'price': {'$lte': 1000}},
                                    {'price': {'$gte': 50}}]},
                'stage': 'COLLSCAN'},
 'isCached': False,
 'memLimit': 33554432,
 'sortPattern': {'metrics.total_units_sold': -1},
 'stage': 'SORT',
 'type': 'simple'}
```

```
product_catalog.drop_index("idx_esr_status_category_units_price")
```

```
product_catalog.create_index(
    [
        ("status", 1),
        ("category", 1),
        ("metrics.total_units_sold", -1),
        ("price", 1)
    ],
    name="idx_pc_esr_status_category_units_price"
)

print("Índice creado: idx_pc_esr_status_category_units_price")

Índice creado: idx_pc_esr_status_category_units_price
```

```
cursor_top_sold_after = product_catalog.find(
    query_catalog_top_sold
).sort("metrics.total_units_sold", -1).hint("idx_pc_esr_status_category_units_price")

result = evaluate_query_productivity(
    "product_catalog - top vendidos usando idx_pc_esr_status_category_units_price",
    cursor_top_sold_after
)

index_evaluation_results.append(result)
```

```
==== product_catalog - top vendidos usando idx_pc_esr_status_category_units_price ====
{'consulta': 'product_catalog - top vendidos usando '
            'idx_pc_esr_status_category_units_price',
 'docsPerReturned': 1.0,
 'executionTimeMillis': 5,
 'keysPerReturned': 1.02,
 'mReturned': 2219,
 'totalDocsExamined': 2219,
 'totalKeysExamined': 2256}

Plan ganador:
{'inputStage': {'direction': 'forward',
                'indexBounds': {'category': [['bed_bath_table', '
                                'bed_bath_table']],
                                'metrics.total_units_sold': [['MaxKey, '
                                                                'MinKey']],
                                'price': [['$0, 1000]]},
                                'status': [['ACTIVE', 'ACTIVE']]},
                'indexName': 'idx_pc_esr_status_category_units_price',
                'indexVersion': 2,
                'isMultiKey': False,
                'isPartial': False,
                'isSparse': False,
                'isUnique': False,
                'keyPattern': {'category': 1,
                              'metrics.total_units_sold': -1,
                              'price': 1,
                              'status': 1},
                'multikeyPaths': {'category': [],
                                  'metrics.total_units_sold': [],
                                  'price': [],
                                  'status': []},
                'stage': 'IXSCAN'},
 'isCached': False,
 'stage': 'FETCH'}
```

```
# Índice parcial para productos activos con reseñas

query_catalog_partial = {
    "category": "bed_bath_table",
    "seller_region": "SP",
    "status": "ACTIVE",
    "ratings.count": {
        "$gte": 10
    },
    "price": {
        "$gte": 50,
        "$lte": 500
    }
}

cursor_partial_before = product_catalog.find(
    query_catalog_partial
).sort("ratings.average", -1).hint({"$natural": 1})

result = evaluate_query_productivity(
    "product_catalog - parcial antes del índice",
    cursor_partial_before
)

index_evaluation_results.append(result)
```

```
==== product_catalog - parcial antes del índice ====
{'consulta': 'product_catalog - parcial antes del índice',
 'docsPerReturned': 274.59,
 'executionTimeMillis': 26,
 'keysPerReturned': 0.0,
 'mReturned': 120,
 'totalDocsExamined': 32951,
 'totalKeysExamined': 0}

Plan ganador:
{'inputStage': {'direction': 'forward',
                'filter': {'$and': [{'category': {'$eq': 'bed_bath_table'}},
                                    {'seller_region': {'$eq': 'SP'}},
                                    {'status': {'$eq': 'ACTIVE'}},
                                    {'price': {'$lte': 500}},
                                    {'price': {'$gte': 50}},
                                    {'ratings.count': {'$gte': 10}}]},
                'stage': 'COLLSCAN'},
 'isCached': False,
 'memLimit': 33554432,
 'sortPattern': {'ratings.average': -1},
 'stage': 'SORT',
 'type': 'simple'}
```

```
product_catalog.create_index(
    [
        ("category", 1),
        ("seller_region", 1),
        ("ratings.average", -1),
        ("price", 1)
    ],
    partialFilterExpression={
        "status": "ACTIVE",
        "ratings.count": {
            "$gte": 10
        }
    },
    name="idx_pc_partial_active_rating_price"
)

print("Índice creado: idx_pc_partial_active_rating_price")

Índice creado: idx_pc_partial_active_rating_price
```

```
cursor_partial_after = product_catalog.find(
    query_catalog_partial
).sort("ratings.average", -1).hint("idx_pc_partial_active_rating_price")

result = evaluate_query_productivity(
    "product_catalog - parcial usando idx_pc_partial_active_rating_price",
    cursor_partial_after
)

index_evaluation_results.append(result)

==== product_catalog - parcial usando idx_pc_partial_active_rating_price ====
{'consulta': 'product_catalog - parcial usando '
            'idx_pc_partial_active_rating_price',
 'docsPerReturned': 1.0,
 'executionTimeMillis': 0,
```



```
'keysPerReturned': 1,4,
'nReturned': 120,
'totalDocsExamined': 120,
'totalKeysExamined': 168}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': [ {'category': [ [ "bed_bath_table",
                                "bed_bath_table" ] ] },
    'price': [ [50, 500] ] ,
    'ratings.average': [ [MaxKey, MinKey] ] ,
    'seller_region': [ [ "SP", "SP" ] ] ],
  'indexName': 'idx_pc_partial_active_rating_price',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': True,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': { 'category': 1,
    'price': 1,
    'ratings.average': -1,
    'seller_region': 1 },
  'multiKeyPaths': { 'category': [ ],
    'price': [ ],
    'ratings.average': [ ],
    'seller_region': [ ] },
  'stage': 'IXSCAN'},
'isCached': False,
'stage': 'FETCH'}
```

Índice de texto para búsqueda full-text

```
product_catalog.create_index(
[
    ("name", "text"),
    ("description", "text"),
    ("tags", "text")
],
name="idx_pc_text_name_description_tags"
)

print("Índice creado: idx_pc_text_name_description_tags")
```

```
query_text = {
    "text": {
        "search": "bed table"
    }
}

cursor_text = product_catalog.find(
    query_text,
    {
        "score": { "$meta": "textScore" },
        "name": 1,
        "category": 1,
        "description": 1,
        "tags": 1
    }
).sort([
    ("score", { "$meta": "textScore" })
])

result = evaluate_query_productivity(
    "product_catalog - full-text usando índice de texto",
    cursor_text
)

index_evaluation_results.append(result)
```

```
==== product_catalog - full-text usando índice de texto ====
{'consulta': 'product_catalog - full-text usando índice de texto',
'docsPerReturned': None,
'executionTimeMillis': 0,
'keysPerReturned': None,
'nReturned': 0,
'totalDocsExamined': 0,
'totalKeysExamined': 0}

Plan ganador:
{'inputStage': {'inputStage': {'indexName': 'idx_pc_text_name_description_tags',
  'indexPrefix': { },
  'inputStage': {'inputStages': [ {'direction': 'backward',
    'indexBounds': { },
    'indexName': 'idx_pc_text_name_description_tags',
    'indexVersion': 2,
    'isMultiKey': True,
    'isPartial': False,
    'isSparse': False,
    'isUnique': False,
    'keyPattern': { '_fts': 'text',
      '_ftsx': 1 },
    'stage': 'IXSCAN' },
    {'direction': 'backward',
      'indexBounds': { },
      'indexName': 'idx_pc_text_name_description_tags',
      'indexVersion': 2,
      'isMultiKey': True,
      'isPartial': False,
      'isSparse': False,
      'isUnique': False,
      'keyPattern': { '_fts': 'text',
        '_ftsx': 1 },
        'stage': 'IXSCAN' } ],
    'stage': 'TEXT_OR' },
    'parsedTextQuery': { 'negatedPhrases': [ ],
      'negatedTerms': [ ],
      'phrases': [ ],
      'terms': [ 'bed', 'tabl' ] },
    'stage': 'TEXT_MATCH',
    'textIndexVersion': 3 },
    'memLimit': 23554432,
    'sortPattern': { '$computed0': { '$meta': 'textScore' } },
    'stage': 'SORT',
    'type': 'default' },
'isCached': False,
'stage': 'PROJECTION_DEFAULT',
'transformBy': { 'category': 1,
  'description': 1,
  'name': 1,
  'score': { '$meta': 'textScore' },
  'tags': 1 }}
```

▼ Coleccion product_reviews

```
# Optimización de product_reviews

sample_review = product_reviews.find_one((), {"product_id": 1})

sample_product_id = sample_review["product_id"]

query_reviews_by_product = {
    "product_id": sample_product_id
}

cursor_reviews_before = product_reviews.find(query_reviews_by_product).hint({"$natural": 1})

result = evaluate_query_productivity(
    "product_reviews - búsqueda por product_id antes/actual",
    cursor_reviews_before
)

index_evaluation_results.append(result)

==== product_reviews - búsqueda por product_id antes/actual ====
{'consulta': 'product_reviews - búsqueda por product_id antes/actual',
'docsPerReturned': 34657,33,
'executionTimeMillis': 60,
'keysPerReturned': 0,0,
'nReturned': 3,
'totalDocsExamined': 102172,
'totalKeysExamined': 0}

Plan ganador:
{'direction': 'forward',
'filter': { 'product_id': { '$eq': 'fd25ab760bfbb13c198fa3b4f1a0cd3' } },
'isCached': False,
'stage': 'COLLSCAN'}
```

```
#product_reviews.drop_index("product_id_1")
product_reviews.drop_index("idx_pr_product_id")
```

```
product_reviews.create_index(
[
    ("product_id", 1)
],
name="idx_pr_product_id"
)

print("Índice creado: idx_pr_product_id")
```

Índice creado: idx_pr_product_id

```
cursor_reviews_after = product_reviews.find(
    query_reviews_by_product
).hint("idx_pr_product_id")

result = evaluate_query_productivity(
    "product_reviews - búsqueda por product_id usando idx_pr_product_id",
    cursor_reviews_after
)

index_evaluation_results.append(result)

==== product_reviews - búsqueda por product_id usando idx_pr_product_id ====
{'consulta': 'product_reviews - búsqueda por product_id usando '
  'idx_pr_product_id',
'docsPerReturned': 1,0,
'executionTimeMillis': 1,
'keysPerReturned': 1,0,
'nReturned': 3,
'totalDocsExamined': 3,
'totalKeysExamined': 3}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': [ {'product_id': [ [ "fd25ab760bfbb13c198fa3b4f1a0cd3",
                                "fd25ab760bfbb13c198fa3b4f1a0cd3" ] ] ],
  'indexName': 'idx_pr_product_id',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': { 'product_id': 1 },
  'multiKeyPaths': { 'product_id': [ ] },
  'stage': 'IXSCAN'},
'isCached': False,
'stage': 'FETCH'}
```

Índice parcial para reseñas verificadas

```
sample_verified_review = product_reviews.find_one({
    "verified_purchase": True
})

sample_verified_product_id = sample_verified_review["product_id"]

query_verified_reviews = {
    "product_id": sample_verified_product_id,
    "verified_purchase": True
}

cursor_verified_before = product_reviews.find(
    query_verified_reviews
).sort("rating", -1)

result = evaluate_query_productivity(
    "product_reviews - reseñas verificadas antes del índice parcial",
    cursor_verified_before
)

index_evaluation_results.append(result)
```

```
==== product_reviews - reseñas verificadas antes del índice parcial ====
{'consulta': 'product_reviews - reseñas verificadas antes del índice parcial',
'docsPerReturned': 1,0,
```


<pre>'executionTimeMillis': 0, 'keysPerReturned': 1.0, 'nReturned': 3, 'totalDocsExamined': 3, 'totalKeysExamined': 3} Plan ganador: {'inputStage': {'filter': {'verified_purchase': {'\$eq': True}}, 'inputStage': {'direction': 'forward', 'indexBounds': {'product_id': [['fd25ab760bfbb13c198fa3b4f1a0cd3', ' ''fd25ab760bfbb13c198fa3b4f1a0cd3']]}, 'indexName': 'product_id_1', 'indexVersion': 2, 'isMultiKey': False, 'isPartial': False, 'isSparse': False, 'isUnique': False, 'keyPattern': {'product_id': 1}, 'multikeyPaths': {'product_id': []}, 'stage': 'IXSCAN'}, 'stage': 'FETCH'}, 'isCached': False, 'memLimit': 33554432, 'sortPattern': {'rating': -1}, 'stage': 'SORT', 'type': 'simple'}</pre>	
<pre>product_reviews.create_index([("product_id", 1), ("rating", -1)], partialFilterExpression={ "verified_purchase": True }, name="idx_pr_partial_verified_product_rating") print("Índice creado: idx_pr_partial_verified_product_rating")</pre>	Índice creado: idx_pr_partial_verified_product_rating
<pre>cursor_verified_after = product_reviews.find(query_verified_reviews).sort("rating", -1).hint("idx_pr_partial_verified_product_rating") result = evaluate_query_productivity("product_reviews - reseñas verificadas usando idx_pr_partial_verified_product_rating", cursor_verified_after) index_evaluation_results.append(result) ==== product_reviews - reseñas verificadas usando idx_pr_partial_verified_product_rating ==== {'consulta': 'product_reviews - reseñas verificadas usando ' 'idx_pr_partial_verified_product_rating', 'docsPerReturned': 1.0, 'executionTimeMillis': 1, 'keysPerReturned': 1.0, 'nReturned': 3, 'totalDocsExamined': 3, 'totalKeysExamined': 3} Plan ganador: {'inputStage': {'direction': 'forward', 'indexBounds': {'product_id': [['fd25ab760bfbb13c198fa3b4f1a0cd3', ' ''fd25ab760bfbb13c198fa3b4f1a0cd3']]}, 'indexName': 'idx_pr_partial_verified_product_rating', 'indexVersion': 2, 'isMultiKey': False, 'isPartial': True, 'isSparse': False, 'isUnique': False, 'keyPattern': {'product_id': 1, 'rating': -1}, 'multikeyPaths': {'product_id': [], 'rating': []}, 'stage': 'IXSCAN'}, 'isCached': False, 'stage': 'FETCH'}</pre>	
<pre># Índice compuesto por calificación y fecha query_low_reviews = { "rating": { "\$lte": 2 } } cursor_low_reviews_before = product_reviews.find(query_low_reviews).sort("created_at", -1) result = evaluate_query_productivity("product_reviews - reseñas negativas recientes antes del índice", cursor_low_reviews_before) index_evaluation_results.append(result) ==== product_reviews - reseñas negativas recientes antes del índice ==== {'consulta': 'product_reviews - reseñas negativas recientes antes del índice', 'docsPerReturned': 6.09, 'executionTimeMillis': 78, 'keysPerReturned': 0.0, 'nReturned': 15275, 'totalDocsExamined': 102172, 'totalKeysExamined': 0} Plan ganador: {'inputStage': {'direction': 'forward', 'filter': {'rating': {'\$lte': 2}}, 'stage': 'COLLSCAN'}, 'isCached': False, 'memLimit': 33554432, 'sortPattern': {'created_at': -1}, 'stage': 'SORT', 'type': 'simple'}</pre>	
<pre>product_reviews.create_index([("rating", 1), ("created_at", -1)], name="idx_pr_rating_created_at") print("Índice creado: idx_pr_rating_created_at")</pre>	Índice creado: idx_pr_rating_created_at
<pre>cursor_low_reviews_after = product_reviews.find(query_low_reviews).sort("created_at", -1).hint("idx_pr_rating_created_at") result = evaluate_query_productivity("product_reviews - reseñas negativas recientes usando idx_pr_rating_created_at", cursor_low_reviews_after) index_evaluation_results.append(result) ==== product_reviews - reseñas negativas recientes usando idx_pr_rating_created_at ==== {'consulta': 'product_reviews - reseñas negativas recientes usando ' 'idx_pr_rating_created_at', 'docsPerReturned': 1.0, 'executionTimeMillis': 74, 'keysPerReturned': 1.0, 'nReturned': 15275, 'totalDocsExamined': 15275, 'totalKeysExamined': 15275} Plan ganador: {'inputStage': {'inputStage': {'direction': 'forward', 'indexBounds': {'created_at': [['MaxKey', ' 'MinKey']]}, 'indexName': 'idx_pr_rating_created_at', 'indexVersion': 2, 'isMultiKey': False, 'isPartial': False, 'isSparse': False, 'isUnique': False, 'keyPattern': {'created_at': -1, 'rating': 1}, 'multikeyPaths': {'created_at': [], 'rating': []}, 'stage': 'IXSCAN'}, 'memLimit': 33554432, 'sortPattern': {'created_at': -1}, 'stage': 'SORT', 'type': 'default'}, 'isCached': False, 'stage': 'FETCH'}</pre>	
▼ Coleccion user_behavior	
<pre># Índice compuesto por usuario y periodo sample_behavior = user_behavior.find_one({}, {"userId": 1, "period": 1}) sample_user_id = sample_behavior["userId"] sample_period = sample_behavior["period"] query_behavior_user_period = { "userId": sample_user_id, "period": sample_period } cursor_behavior_before = user_behavior.find(query_behavior_user_period) result = evaluate_query_productivity("user_behavior - consulta por usuario y periodo antes del índice", cursor_behavior_before) index_evaluation_results.append(result) ==== user_behavior - consulta por usuario y periodo antes del índice ==== {'consulta': 'user_behavior - consulta por usuario y periodo antes del índice', 'docsPerReturned': 1.0, 'executionTimeMillis': 0, 'keysPerReturned': 1.0, 'nReturned': 1, 'totalDocsExamined': 1, 'totalKeysExamined': 1} Plan ganador: {'filter': {'userId': {'\$eq': '3ce436f183e6e0877b285a838db11a'}}}, 'inputStage': {'direction': 'forward', 'indexBounds': {'period': [['2017-09', '2017-09']], 'userId': [['7769316519409380230', ''7769316519409380230']]}, 'indexName': 'userId_hashed_period_1', 'indexVersion': 2, 'isMultiKey': False, 'isPartial': False, 'isSparse': False, 'isUnique': False, 'keyPattern': {'period': 1, 'userId': 'hashed'}, 'stage': 'IXSCAN'}, 'isCached': False, 'stage': 'FETCH'}</pre>	
<pre>#user_behavior.drop_index("idx_user_behavior_lookup") user_behavior.drop_index("idx_ub_user_period")</pre>	
<pre>user_behavior.create_index([("userId", 1), ("period", 1)], name="idx_ub_user_period") print("Índice creado: idx_ub_user_period") Índice creado: idx_ub_user_period</pre>	


```
cursor_behavior_after = user_behavior.find(
    query_behavior_user_period
).hint(("idx_ub_user_period")

result = evaluate_query_productivity(
    "user_behavior - consulta por usuario y periodo usando idx_ub_user_period",
    cursor_behavior_after
)

index_evaluation_results.append(result)

==== user_behavior - consulta por usuario y periodo usando idx_ub_user_period ====
{'consulta': 'user_behavior - consulta por usuario y periodo usando '
  'idx_ub_user_period',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 2,
  'keysPerReturned': 1.0,
  'mReturned': 1,
  'totalDocsExamined': 1,
  'totalKeysExamined': 1}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': {'period': [['2017-09', '2017-09']],
    'userId': [['3c436f183e68e07877b285a838db11a', '
      '3ce436f183e68e07877b285a838db11a']]}],
  'indexName': 'idx_ub_user_period',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'period': 1, 'userId': 1},
  'multikeyPaths': {'period': [], 'userId': []},
  'stage': 'IXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

```
# Índice multikey para eventos por tipo

query_behavior_purchase = {
    "events.type": "PURCHASE"
}

cursor_purchase_before = user_behavior.find(query_behavior_purchase)

result = evaluate_query_productivity(
    "user_behavior - eventos PURCHASE antes del índice",
    cursor_purchase_before
)

index_evaluation_results.append(result)
```

```
==== user_behavior - eventos PURCHASE antes del índice ====
{'consulta': 'user_behavior - eventos PURCHASE antes del índice',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 176,
  'keysPerReturned': 0.0,
  'mReturned': 112650,
  'totalDocsExamined': 112650,
  'totalKeysExamined': 0}

Plan ganador:
{'direction': 'forward',
  'filter': {'events.type': {'$eq': 'PURCHASE'}}},
  'isCached': False,
  'stage': 'COLLSCAN'}
```

```
user_behavior.create_index(
    [
        ("events.type", 1),
        ("period", 1)
    ],
    name="idx_ub_event_type_period"
)

print("Índice creado: idx_ub_event_type_period")

Índice creado: idx_ub_event_type_period
```

```
cursor_purchase_after = user_behavior.find(
    query_behavior_purchase
).hint(("idx_ub_event_type_period")

result = evaluate_query_productivity(
    "user_behavior - eventos PURCHASE usando idx_ub_event_type_period",
    cursor_purchase_after
)

index_evaluation_results.append(result)

==== user_behavior - eventos PURCHASE usando idx_ub_event_type_period ====
{'consulta': 'user_behavior - eventos PURCHASE usando idx_ub_event_type_period',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 190,
  'keysPerReturned': 1.0,
  'mReturned': 112650,
  'totalDocsExamined': 112650,
  'totalKeysExamined': 112650}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': {'events.type': [['"PURCHASE", "PURCHASE"]],
    'period': [['MinKey', MaxKey]]}],
  'indexName': 'idx_ub_event_type_period',
  'indexVersion': 2,
  'isMultiKey': True,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'events.type': 1, 'period': 1},
  'multikeyPaths': {'events.type': ['events'], 'period': []},
  'stage': 'IXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

```
# Índice parcial para eventos de compra

query_behavior_purchase_period = {
    "events.type": "PURCHASE",
    "period": sample_period
}

cursor_purchase_period_before = user_behavior.find(query_behavior_purchase_period)

result = evaluate_query_productivity(
    "user_behavior - PURCHASE por periodo antes del índice parcial",
    cursor_purchase_period_before
)

index_evaluation_results.append(result)

==== user_behavior - PURCHASE por periodo antes del índice parcial ====
{'consulta': 'user_behavior - PURCHASE por periodo antes del índice parcial',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 0,
  'keysPerReturned': 1.0,
  'mReturned': 4831,
  'totalDocsExamined': 4831,
  'totalKeysExamined': 4831}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': {'events.type': [['"PURCHASE", "PURCHASE"]],
    'period': [['"2017-09", "2017-09"]]}],
  'indexName': 'idx_ub_event_type_period',
  'indexVersion': 2,
  'isMultiKey': True,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'events.type': 1, 'period': 1},
  'multikeyPaths': {'events.type': ['events'], 'period': []},
  'stage': 'IXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

```
user_behavior.create_index(
    [
        ("period", 1),
        ("userId", 1)
    ],
    partialFilterExpression={
        "events.type": "PURCHASE"
    },
    name="idx_ub_partial_purchase_period_user"
)

print("Índice creado: idx_ub_partial_purchase_period_user")

Índice creado: idx_ub_partial_purchase_period_user
```

```
cursor_purchase_period_after = user_behavior.find(
    query_behavior_purchase_period
).hint(("idx_ub_partial_purchase_period_user")

result = evaluate_query_productivity(
    "user_behavior - PURCHASE por periodo usando idx_ub_partial_purchase_period_user",
    cursor_purchase_period_after
)

index_evaluation_results.append(result)

==== user_behavior - PURCHASE por periodo usando idx_ub_partial_purchase_period_user ====
{'consulta': 'user_behavior - PURCHASE por periodo usando '
  'idx_ub_partial_purchase_period_user',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 14,
  'keysPerReturned': 1.0,
  'mReturned': 4831,
  'totalDocsExamined': 4831,
  'totalKeysExamined': 4831}

Plan ganador:
{'inputStage': {'direction': 'forward',
  'indexBounds': {'period': [['"2017-09", "2017-09"]],
    'userId': [['MinKey', MaxKey]]}],
  'indexName': 'idx_ub_partial_purchase_period_user',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': True,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'period': 1, 'userId': 1},
  'multikeyPaths': {'period': [], 'userId': []},
  'stage': 'IXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

▼ Optimización de sellers

```
# Índice por región

sample_seller = sellers.find_one({}, {"region": 1})

sample_region = sample_seller["region"]

query_seller_region = {
    "region": sample_region
}

cursor_seller_region_before = sellers.find(query_seller_region)

result = evaluate_query_productivity(
    "sellers - consulta por región antes del índice",
    cursor_seller_region_before
)

index_evaluation_results.append(result)
```

```
==== sellers - consulta por región antes del índice ====
{'consulta': 'sellers - consulta por región antes del índice',
  'docsPerReturned': 1.0,
  'executionTimeMillis': 2,
  'keysPerReturned': 0.0,
  'mReturned': 1849,
  'totalDocsExamined': 3095,
  'totalKeysExamined': 0}
```



```
Plan ganador:
{'direction': 'forward',
 'filter': {'region': {'$eq': 'SP'}},
 'isCached': False,
 'stage': 'COLLSCAN'}
```

```
sellers.create_index(
    [
        ("region", 1)
    ],
    name="idx_sellers_region"
)

print("índice creado: idx_sellers_region")

índice creado: idx_sellers_region
```

```
cursor_seller_region_after = sellers.find(
    query_seller_region
).hint('idx_sellers_region')

result = evaluate_query_productivity(
    cursors - consulta_por_region usando idx_sellers_region',
    cursor_seller_region_after
)

index_evaluation_results.append(result)

==== sellers - consulta por region usando idx_sellers_region ====
{'consulta': 'sellers - consulta por region usando idx_sellers_region',
 'docsPerReturned': 1.0
 'executionTimeInMillis': 3,
 'keysPerReturned': 1.0,
 'returned': 1849,
 'totalDocsExamined': 1849,
 'totalKeysExamined': 1849}

Plan gadgeto:
{
  'inputStage': {
    'direction': 'forward',
    'indexBounds': {'region': [['SP', 'SP']]},
    'indexName': 'idx_sellers_region',
    'indexVersion': 2,
    'isMultiKey': False,
    'isPartial': False,
    'isSparse': False,
    'isUnique': False,
    'keyPatterns': {'region': 1},
    'multiKeyPaths': {'region': {}},
    'stage': 'IXSCAN'
  },
  'isCached': False,
  'stage': 'FETCH'
}
```

```
# Índice compuesto por región y ciudad

sample_seller_city = sellers.find_one(), ("region": 1, "city": 1)

query_seller_region_city = {
    "region": sample_seller_city["region"],
    "city": sample_seller_city["city"]
}

cursor_seller_region_city_before = sellers.find(query_seller_region_city)

result = evaluta_query_productivity(
    "sellers - consulta por región y ciudad antes del índice",
    cursor_seller_region_city_before
)

index_evaluation_results.append(result)

=== sellers - consulta por región y ciudad antes del índice ===
{'consulta': 'sellers - consulta por región y ciudad antes del índice',
 'docsPerReturned': 45,
 'executionTimeInMs': 2,
 'keysPerReturned': 45,
 'Returned': 41,
 'totalDocsExamined': 1849,
 'totalKeysExamined': 1849}
```

```
'filter': {'city': {'$eq': 'campinas'}},
'inputStage': {'$direction': 'forward',
'indexBounds': {'$region': [{'$eq': 'idx_sellers_region'}]},
'indexName': 'idx_sellers_region',
'indexVersion': 2,
'$multiKey': False,
'$isPartial': False,
'$isSparse': False,
'$isUnique': False,
'keyPattern': {'$region': 1},
'multiKeyPaths': {'$region': []},
'stage': '$XSCAN'},
'isCached': False,
'stage': '$FETCH'}

sellers.create_index(
    [
        ("region", 1),
        ("city", 1)
    ],
    name='idx_sellers_region_city'
)

print("Índice creado: idx_sellers_region_city")

Índice creado: idx_sellers_region_city
```

```
cursor_seller_region_city_after = sellers.find(
    query_seller_region_city
).hint('idx_sellers_region_city')

result = evaluate_query_productivity(
    'sellers - consulta por region y ciudad usando idx_sellers_region_city',
    cursor_seller_region_city_after
)

index_evaluation_results.append(result)

===== sellers - consulta por region y ciudad usando idx_sellers_region_city =====
{'consulta': 'sellers - consulta por region y ciudad usando ',
 'idx_sellers_region_city',
 'docsPerReturned': 1.0,
 'executionTimeInMillis': 1,
 'keysPerReturned': 1.0,
 'returned': 41,
 'totalDocsExamined': 41,
 'totalKeysExamined': 41}

Plan ganador:
{'inputStage': {'indexName': 'forward',
 'region': {'city': [{'campinas', 'campinas']}],
 'idx_sellers_region_city',
 'indexVersion': 2,
 'isMultiKey': False,
 'isPartial': False,
 'isSparse': False,
 'isUnique': False,
 'keyPattern': {'city': 1, 'region': 1},
 'multikeyPaths': {'city': 1, 'region': []},
 'stage': 'IXSCAN'},
 'isCached': False,
 'stage': 'FEIGH'}
```

Resultados Finales

```
df_index_evaluation = pd.DataFrame(index_evaluation_results)
df_index_evaluation.sort_values(
    by=["docsPerReturned", "executionTimeMillis"],
    ascending=[False, False]
)
df_index_evaluation.to_csv("evaluacion_productividad_indices_economfy.csv", index=False)
print("Archivo generado: evaluacion_productividad_indices_economfy.csv")

df_index_evaluation
```


Archivo generado: evaluacion_productividad_indices_ecommfy.csv

		consulta	mReturned	totalDocsExamined	totalKeysExamined	executionTimeMillis	docsPerReturned	keysPerReturned	
0	product_catalog - ESR antes del índice	1854		1854	1942	10	1.00	1.05	
1	product_catalog - ESR antes del índice	1854		2193	2283	12	1.18	1.23	
2	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	6	1.00	1.05	
3	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
4	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
5	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
6	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	5	1.00	1.05	
7	product_catalog - ESR antes del índice	1854		2193	2283	12	1.18	1.23	
8	product_catalog - ESR antes del índice	1854		1854	1942	10	1.00	1.05	
9	product_catalog - ESR antes del índice	1854		1854	1942	8	1.00	1.05	
10	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
11	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
12	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
13	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
14	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
15	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	5	1.00	1.05	
16	product_catalog - ESR antes del índice	1854		1854	1942	10	1.00	1.05	
17	product_catalog - ESR antes del índice	1854		2193	2283	12	1.18	1.23	
18	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	6	1.00	1.05	
19	product_catalog - ESR antes del índice	1854		1854	1942	9	1.00	1.05	
20	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	5	1.00	1.05	
21	product_catalog - ESR antes del índice	1854		2193	2283	12	1.18	1.23	
22	product_catalog - ESR antes del índice	1854		2193	2283	13	1.18	1.23	
23	product_catalog - ESR antes del índice	1854		32951	0	30	17.77	0.00	
24	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	6	1.00	1.05	
25	product_catalog - top vendidos antes del índice	2219		32951	0	30	14.85	0.00	
26	product_catalog - top vendidos usando idx_pc_e...	2219		2219	2256	7	1.00	1.02	
27	product_catalog - parcial antes del índice	120		1854	1942	41	15.45	16.18	
28	product_catalog - parcial antes del índice	120		32951	0	27	274.59	0.00	
29	product_catalog - parcial usando idx_pc_partia...	120		120	168	1	1.00	1.40	
30	product_catalog - full-text usando índice de t...	0		0	0	2	NaN	NaN	
31	product_reviews - búsqueda por product_id ante...	3		3	3	3	1.00	1.00	
32	product_reviews - búsqueda por product_id ante...	3		102172	0	97	34057.33	0.00	
33	product_reviews - reseñas verificadas antes de...	3		3	3	0	1.00	1.00	
34	product_reviews - reseñas verificadas usando l...	3		3	3	1	1.00	1.00	
35	product_reviews - reseñas negativas recientes ...	15275		102172	0	78	6.69	0.00	
36	product_reviews - reseñas negativas recientes ...	15275		15275	15275	74	1.00	1.00	
37	user_behavior - consulta por usuario y periodo...	1		1	1	6	1.00	1.00	
38	product_catalog - ESR antes del índice	1854		32951	0	45	17.77	0.00	
39	product_catalog - ESR usando idx_pc_esr_catego...	1854		1854	1942	6	1.00	1.05	
40	product_catalog - top vendidos antes del índice	2219		32951	0	29	14.85	0.00	
41	product_catalog - top vendidos usando idx_pc_e...	2219		2219	2256	5	1.00	1.02	
42	product_catalog - parcial antes del índice	120		32951	0	26	274.59	0.00	
43	product_catalog - parcial usando idx_pc_partia...	120		120	168	0	1.00	1.40	
44	product_catalog - full-text usando índice de t...	0		0	0	0	NaN	NaN	
45	product_reviews - búsqueda por product_id ante...	3		102172	0	60	34057.33	0.00	
46	product_reviews - búsqueda por product_id ante...	3		102172	0	60	34057.33	0.00	
47	user_behavior - consulta por usuario y periodo...	1		1	1	0	1.00	1.00	
48	user_behavior - eventos PURCHASE antes del índice	112650		112650	0	176	1.00	0.00	
49	user_behavior - eventos PURCHASE usando idx_ub...	112650		112650	112650	190	1.00	1.00	
50	user_behavior - PURCHASE por periodo antes del...	4831		4831	4831	8	1.00	1.00	
51	user_behavior - PURCHASE por periodo usando id...	4831		4831	4831	14	1.00	1.00	
52	sellers - consulta por región antes del índice	1849		3095	0	2	1.67	0.00	
53	sellers - consulta por región usando idx_selle...	1849		1849	1849	3	1.00	1.00	
54	sellers - consulta por región y ciudad antes d...	41		1849	1849	2	45.10	45.10	
55	sellers - consulta por región y ciudad antes d...	41		1849	1849	2	45.10	45.10	
56	sellers - consulta por región y ciudad usando ...	41		41	41	1	1.00	1.00	
57	product_reviews - búsqueda por product_id usan...	3		3	3	1	1.00	1.00	
58	user_behavior - consulta por usuario y periodo...	1		1	1	2	1.00	1.00	

Next steps: [Generate code with df_index_evaluation](#) [New interactive sheet](#)

5. Optimización del aggregation pipeline

En esta sección se desarrolla un pipeline complejo sobre las colecciones `product_catalog` y `product_reviews`.

El objetivo es analizar el desempeño comercial de productos agrupados por categoría y región del vendedor, incorporando información de reseñas mediante `$lookup`.

El pipeline cumple con la complejidad mínima requerida porque incluye más de cinco stages y utiliza:

- `$match` para filtrado.
- `$lookup` para relacionar productos con reseñas.
- `$unwind` para procesar reseñas individualmente.
- `$group` para agregación.
- `$addFields` para transformación y cálculo de métricas.
- `$project` para proyección de campos.
- `$sort` para ordenamiento.
- `$limit` para limitar resultados.

También se utiliza `allowDiskUse=True`, lo cual permite que MongoDB use disco temporal cuando el pipeline supera el límite de memoria disponible para agregaciones.

Crear índices de apoyo para el pipeline

```
# Índices de apoyo para el pipeline de agregación

def create_index_if_not_exists(collection, keys, name):
    existing_names = [idx["name"] for idx in collection.list_indexes()]

    if name in existing_names:
        print(f"El índice ya existe: {name}")
    else:
        collection.create_index(keys, name=name)
        print(f"Índice creado: {name}")

create_index_if_not_exists(
    product_catalog,
    [
        ("status", 1),
        ("category", 1),
        ("seller_region", 1),
        ("price", 1)
    ],
    "idx_pipeline_match_catalog"
)

# No se crea porque ya existe el mismo índice con otro nombre
print("El índice para product_reviews.product_id ya existe: idx_pr_product_id")

El índice ya existe: idx_pipeline_match_catalog
El índice para product_reviews.product_id ya existe: idx_pr_product_id
```

Pipeline original no optimizado

```
pipeline_original = [
    {
        "$match": {
            "status": "ACTIVE",
            "category": {
                "$in": [
                    "bed_bath_table",
                    "health_beauty",
                    "sports_leisure"
                ]
            },
            "price": {
                "$gte": 20,
                "$lte": 1000
            }
        }
    },
    {
        "$lookup": {
            "from": "product_reviews",
            "localField": "_id",
            "foreignField": "product_id",
            "as": "reviews"
        }
    },
    {
        "$unwind": {
            "path": "$reviews",
            "preserveNullAndEmptyArrays": True
        }
    },
    {
        "$group": {
            "_id": {
                "category": "$category",
                "seller_region": "$seller_region"
            },
            "total_products": {
                "$sum": 1
            },
            "avg_price": {
                "$avg": "$price"
            },
            "avg_rating": {
                "$avg": "$ratings.average"
            },
            "total_units_sold": {
                "$sum": "$metrics.total_units_sold"
            },
            "total_revenue": {
                "$sum": "$metrics.total_revenue"
            },
            "review_count": {
                "$sum": {
                    "$cond": [
                        {
                            "$ifNull": [
                                "$reviews._id",
                                False
                            ]
                        },
                        1,
                        0
                    ]
                }
            }
        }
    }
]
```



```
    }
  }
},
{
  "$addFields": {
    "sales_score": {
      "$multiply": [
        "$avg_rating",
        "$total_units_sold"
      ]
    }
  }
},
{
  "$project": {
    "_id": 0,
    "category": "$_id.category",
    "seller_region": "$_id.seller_region",
    "total_products": 1,
    "avg_price": {
      "$round": [
        "$avg_price",
        2
      ]
    },
    "avg_rating": {
      "$round": [
        "$avg_rating",
        2
      ]
    },
    "total_units_sold": 1,
    "total_revenue": {
      "$round": [
        "$total_revenue",
        2
      ]
    },
    "review_count": 1,
    "sales_score": {
      "$round": [
        "$sales_score",
        2
      ]
    }
  }
},
{
  "$sort": {
    "sales_score": -1
  }
},
{
  "$limit": 10
}
```

```
result_original = list(
  product_catalog.aggregate(
    pipeline_original,
    allowDiskUse=True
  )
)

for item in result_original:
  print(item)

{'total_products': 8659, 'total_units_sold': 487837, 'review_count': 8646, 'category': 'bed_bath_table', 'seller_region': 'SP', 'avg_price': 96.8, 'avg_rating': 3.91, 'total_revenue': 36838987.88, 'sales_score': 1595476.34}
{'total_products': 4495, 'total_units_sold': 352319, 'review_count': 5489, 'category': 'health_beauty', 'seller_region': 'SP', 'avg_price': 114.96, 'avg_rating': 4.14, 'total_revenue': 32459890.88, 'sales_score': 1457525.43}
{'total_products': 4628, 'total_units_sold': 65487, 'review_count': 4613, 'category': 'sports_leisure', 'seller_region': 'SP', 'avg_price': 111.8, 'avg_rating': 4.11, 'total_revenue': 5296654.56, 'sales_score': 269465.68}
{'total_products': 194, 'total_units_sold': 45748, 'review_count': 394, 'category': 'health_beauty', 'seller_region': 'MA', 'avg_price': 98.3, 'avg_rating': 4.8, 'total_revenue': 4339997.52, 'sales_score': 183213.77}
{'total_products': 588, 'total_units_sold': 28148, 'review_count': 580, 'category': 'health_beauty', 'seller_region': 'PR', 'avg_price': 192.35, 'avg_rating': 4.25, 'total_revenue': 8763494.97, 'sales_score': 119679.07}
{'total_products': 981, 'total_units_sold': 16487, 'review_count': 981, 'category': 'health_beauty', 'seller_region': 'RJ', 'avg_price': 156.84, 'avg_rating': 4.23, 'total_revenue': 2416729.01, 'sales_score': 69803.03}
{'total_products': 512, 'total_units_sold': 14396, 'review_count': 512, 'category': 'bed_bath_table', 'seller_region': 'SC', 'avg_price': 105.2, 'avg_rating': 3.72, 'total_revenue': 1923771.71, 'sales_score': 53608.08}
{'total_products': 1123, 'total_units_sold': 11864, 'review_count': 1130, 'category': 'sports_leisure', 'seller_region': 'PA', 'avg_price': 121.76, 'avg_rating': 4.21, 'total_revenue': 1393997.37, 'sales_score': 49088.94}
{'total_products': 543, 'total_units_sold': 11686, 'review_count': 543, 'category': 'sports_leisure', 'seller_region': 'SC', 'avg_price': 110.11, 'avg_rating': 4.2, 'total_revenue': 786790.17, 'sales_score': 49107.24}
{'total_products': 132, 'total_units_sold': 7246, 'review_count': 132, 'category': 'health_beauty', 'seller_region': 'DF', 'avg_price': 108.96, 'avg_rating': 4.13, 'total_revenue': 606117.02, 'sales_score': 29943.55}
```

▼ Pipeline optimizado

```
pipeline_optimized = [
  {
    "$match": {
      "status": "ACTIVE",
      "category": {
        "$in": [
          "bed_bath_table",
          "health_beauty",
          "sports_leisure"
        ]
      },
      "price": {
        "$gte": 20,
        "$lte": 1000
      }
    },
  },
  {
    "$project": {
      "_id": 1,
      "category": 1,
      "seller_region": 1,
      "price": 1,
      "ratings.average": 1,
      "metrics.total_units_sold": 1,
      "metrics.total_revenue": 1
    }
  },
  {
    "$lookup": {
      "from": "product_reviews",
      "localField": "_id",
      "foreignField": "product_id",
      "as": "reviews"
    }
  },
  {
    "$addFields": {
      "review_count": {
        "$size": "$reviews"
      }
    }
  },
  {
    "$group": {
      "_id": {
        "category": "$category",
        "seller_region": "$seller_region"
      },
      "total_products": {
        "$sum": 1
      },
      "avg_price": {
        "$avg": "$price"
      },
      "avg_rating": {
        "$avg": "$ratings.average"
      },
      "total_units_sold": {
        "$sum": "$metrics.total_units_sold"
      },
      "total_revenue": {
        "$sum": "$metrics.total_revenue"
      },
      "review_count": {
        "$sum": "$review_count"
      }
    }
  },
  {
    "$addFields": {
      "sales_score": {
        "$multiply": [
          "avg_rating",
          "$total_units_sold"
        ]
      }
    }
  },
  {
    "$project": {
      "_id": 0,
      "category": "$_id.category",
      "seller_region": "$_id.seller_region",
      "total_products": 1,
      "avg_price": {
        "$round": [
          "$avg_price",
          2
        ]
      },
      "avg_rating": {
        "$round": [
          "$avg_rating",
          2
        ]
      },
      "total_units_sold": 1,
      "total_revenue": {
        "$round": [
          "$total_revenue",
          2
        ]
      },
      "review_count": 1,
      "sales_score": {
        "$round": [
          "$sales_score",
          2
        ]
      }
    }
  },
  {
    "$sort": {
      "sales_score": -1
    }
  },
  {
    "$limit": 10
  }
]
```

```
result_optimized = list(
  product_catalog.aggregate(
    pipeline_optimized,
    allowDiskUse=True
  )
)

for item in result_optimized:
  print(item)

{'total_products': 2448, 'total_units_sold': 9383, 'review_count': 8646, 'category': 'bed_bath_table', 'seller_region': 'SP', 'avg_price': 108.02, 'avg_rating': 3.85, 'total_revenue': 891621.61, 'sales_score': 36155.43}
{'total_products': 4486, 'total_units_sold': 5862, 'review_count': 5489, 'category': 'health_beauty', 'seller_region': 'SP', 'avg_price': 128.3, 'avg_rating': 4.14, 'total_revenue': 664531.03, 'sales_score': 24288.64}
{'total_products': 1657, 'total_units_sold': 5867, 'review_count': 4613, 'category': 'sports_leisure', 'seller_region': 'SP', 'avg_price': 125.18, 'avg_rating': 4.13, 'total_revenue': 546783.55, 'sales_score': 20830.83}
{'total_products': 436, 'total_units_sold': 1324, 'review_count': 1230, 'category': 'sports_leisure', 'seller_region': 'PR', 'avg_price': 123.74, 'avg_rating': 4.16, 'total_revenue': 161893.46, 'sales_score': 5510.94}
{'total_products': 292, 'total_units_sold': 1894, 'review_count': 981, 'category': 'health_beauty', 'seller_region': 'RJ', 'avg_price': 164.02, 'avg_rating': 4.27, 'total_revenue': 159476.78, 'sales_score': 4502.46}
{'total_products': 157, 'total_units_sold': 618, 'review_count': 543, 'category': 'sports_leisure', 'seller_region': 'SC', 'avg_price': 122.08, 'avg_rating': 4.15, 'total_revenue': 63876.95, 'sales_score': 2591.53}
{'total_products': 162, 'total_units_sold': 614, 'review_count': 580, 'category': 'health_beauty', 'seller_region': 'PR', 'avg_price': 139.92, 'avg_rating': 4.19, 'total_revenue': 116396.95, 'sales_score': 2572.62}
{'total_products': 137, 'total_units_sold': 541, 'review_count': 512, 'category': 'bed_bath_table', 'seller_region': 'SC', 'avg_price': 81.15, 'avg_rating': 3.55, 'total_revenue': 56758.67, 'sales_score': 1920.12}
{'total_products': 148, 'total_units_sold': 455, 'review_count': 405, 'category': 'sports_leisure', 'seller_region': 'MG', 'avg_price': 86.43, 'avg_rating': 3.87, 'total_revenue': 35945.82, 'sales_score': 1766.53}
{'total_products': 139, 'total_units_sold': 398, 'review_count': 371, 'category': 'health_beauty', 'seller_region': 'MG', 'avg_price': 115.79, 'avg_rating': 4.17, 'total_revenue': 56928.25, 'sales_score': 1626.3}
```

▼ Medición de rendimiento

